

deep learning

机器学习

1.

Looking for Function

1. Regression : outputs a scalar 回归分析
2. classification : Alpha Go

Structure Learning

create something with structure (image, document)

2.how to find a function

1. Function with unknown parameters 未知参数的函数 ----- model

Weight 权重 bias 偏置 $y = b + wx$

2. Define Loss 损失函数

输入: b , w 就是参数

作用: how good a set of values is. From training Data

3. Optimization

Gradient Descent 梯度下降

Learning Rate 学习率



hyperparameters 超参数 机器学习里面自己要设定的东西

Local minima Global minima



Does local minima truly cause the problem?

3.linear models are too simple

来自于 modal 的限制 叫做 model 的 bias

Piecewise linear 分段线性

Deep Learning 叠了很多层的神经网络

📦 理论上来说同一层叠足够多的sigmoid/ReLU 就可以拟合任何函数
那为啥需要训练更深层次的神经网络呢？

overfitting 过拟合 训练资料上变好 但是在推理时变差

4.model bias VS optimization Issue

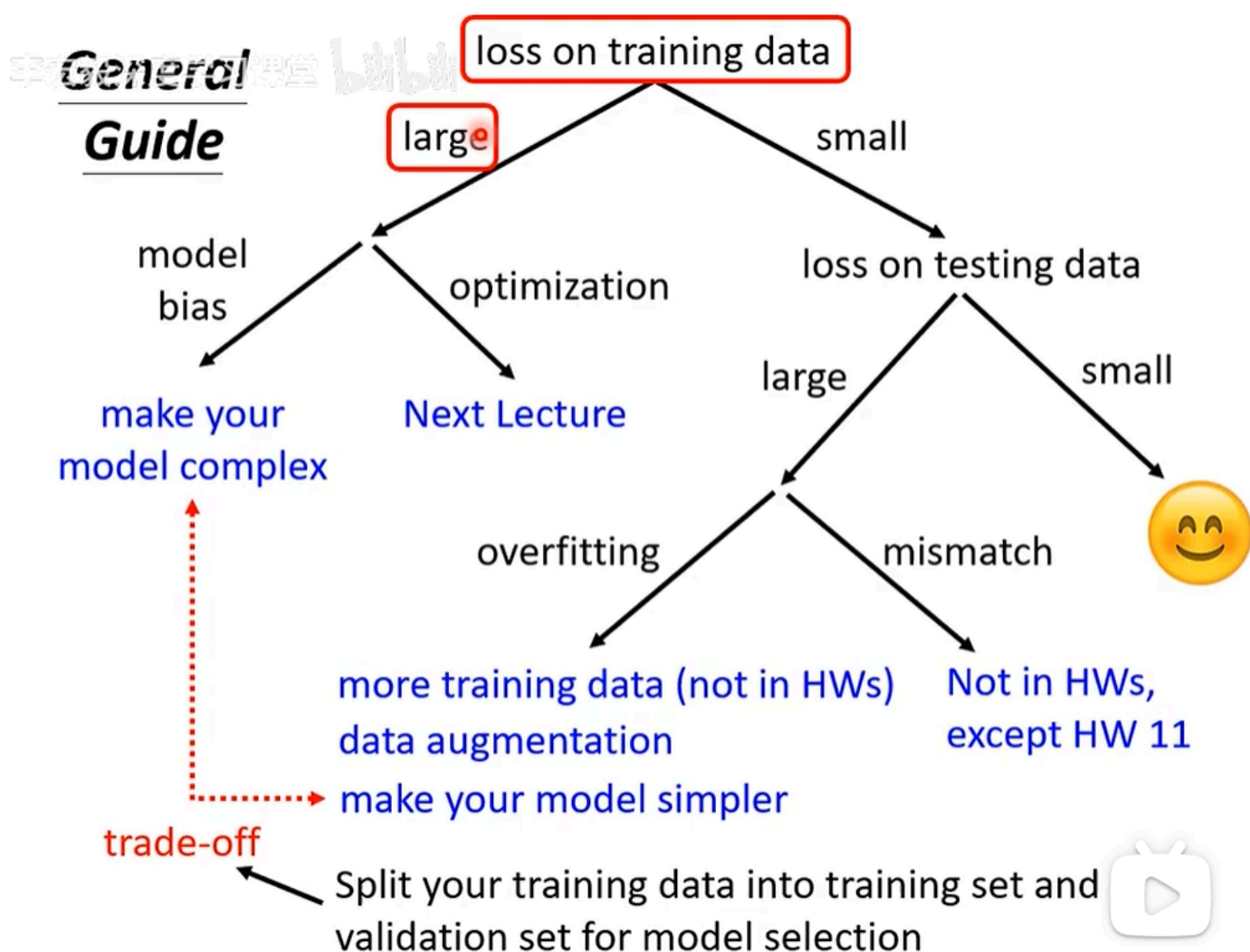


图4.1 General Guide

Model Bias v.s. Optimization Issue

- Gaining the insights from comparison

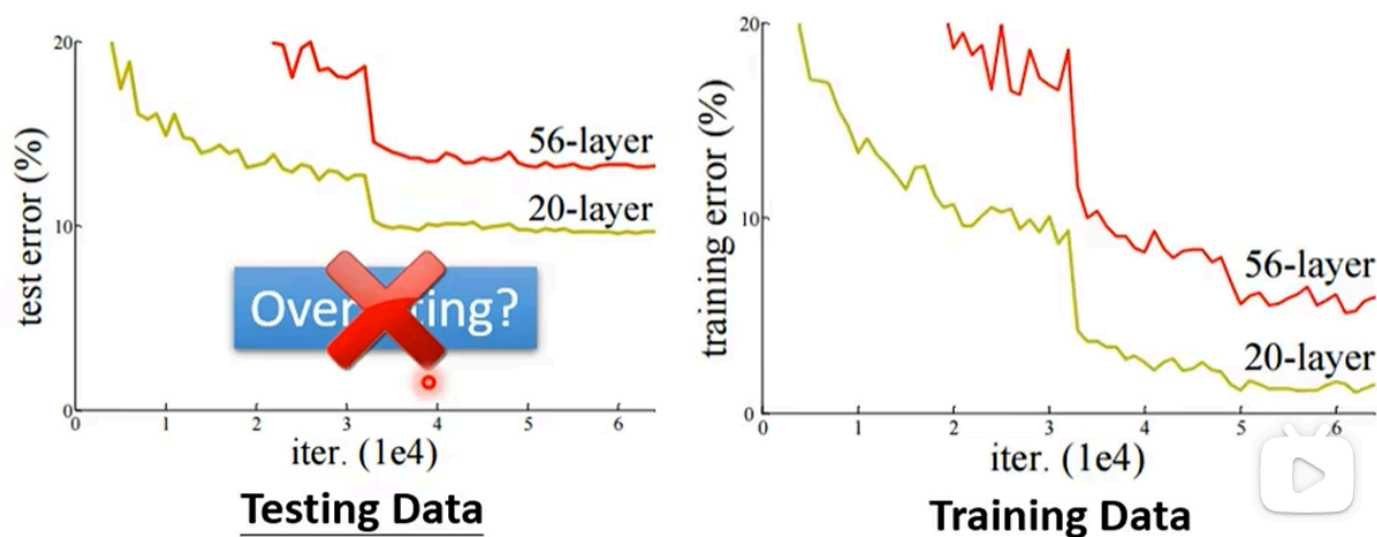


图4.2 模型问题还是优化问题？如何判断

看训练时候的Loss

如何知道自己的模型够不够好

可以先试一下浅层的model

5.如何防止过拟合

- Less Parameters, sharing parameters
- Less features
- Early stopping
- Regularization
- Dropout

Opimization 问题

1.是local minima 还是 saddle point

At critical point:

Hessian

$$L(\theta) \approx L(\theta') + \frac{1}{2} (\theta - \theta')^T H (\theta - \theta')$$

For all v

$v^T H v > 0 \implies \text{Around } \theta': L(\theta) > L(\theta') \implies \text{Local minima}$
 $= H \text{ is positive definite} = \text{All eigen values are positive.} \uparrow$

For all v

$v^T H v < 0 \implies \text{Around } \theta': L(\theta) < L(\theta') \implies \text{Local maxima}$

Sometimes $v^T H v > 0$, sometimes $v^T H v < 0 \implies \text{Saddle point}$

图1.1 这部分线性代数知识刚刚学过

2.为什么使用batch

如果不使用batch，需要将所有训练资料都看完才更新一次loss。

2.1 Small Batch vs Large Batch

- Small Batch : Short time for cooldown, but noisy
- Large Batch : Long time for cooldown, but powerful

考虑到 GPU 并行计算，所以实际上在一定范围内，Large Batch 消耗时间会少一点。

✨ Batch size 居然会影响训练结果？

大的 Batch size 可能导致 optimization 出现问题。

Small Batch v.s. Large Batch

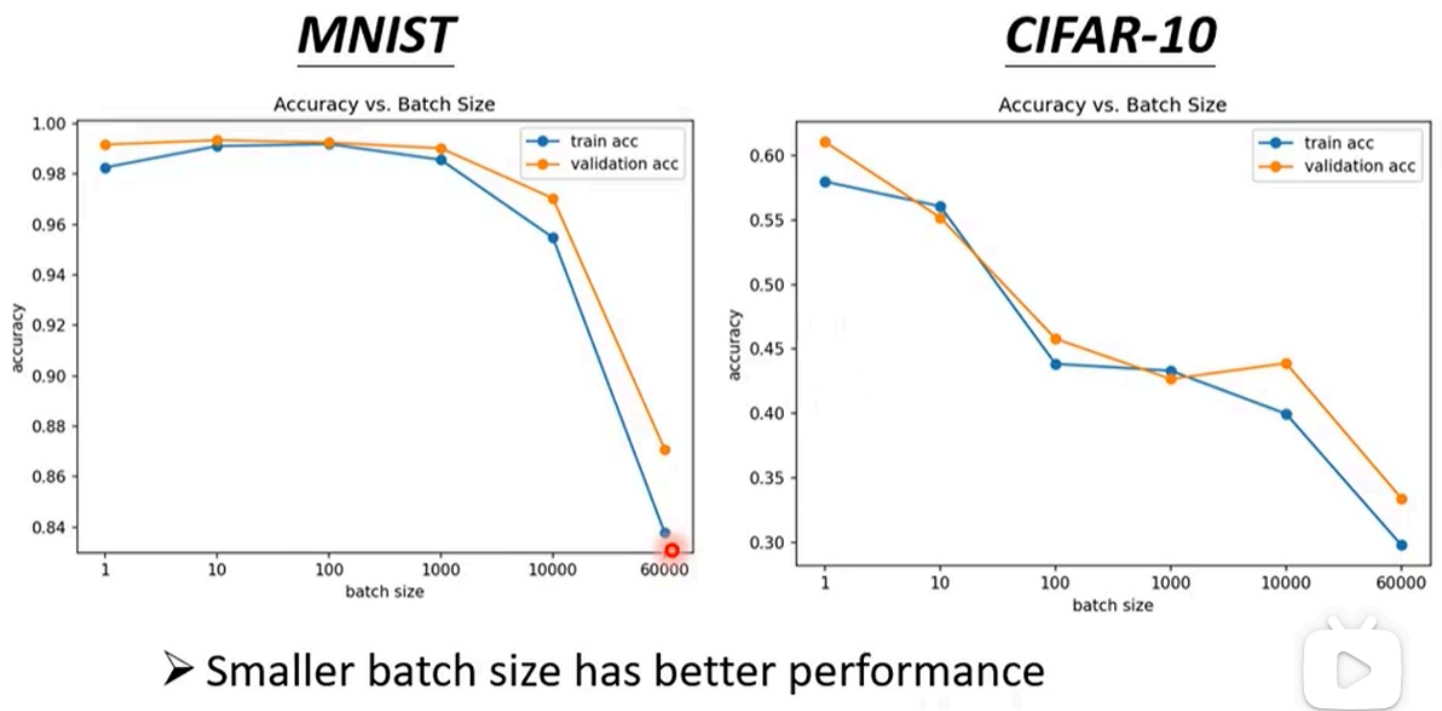


图2.1.1 Batch size 对训练结果的影响

还有更神奇的事情，Batch size 不同时候，即便把 Training Acc 训练到差不多结果 Testing Acc 也会有差异。

Smaller batch size 在训练和测试结果上是更优秀的，有一些方法也可以去解决 Large Batch size 的问题

3.Gradient Descent + momentum

更新梯度时候参考上一次梯度更新的方向。

4.Training can be difficult without critical points

学习率过大会卡在峡谷两边，过小会难以训练，拟合很慢。

为每一个参数都搞一个学习率

Root Mean Square

$$\theta_i^{t+1} \leftarrow \theta_i^t - \boxed{\frac{\eta}{\sigma_i^t}} g_i^t$$

$$\theta_i^1 \leftarrow \theta_i^0 - \frac{\eta}{\sigma_i^0} g_i^0 \quad \sigma_i^0 = \sqrt{(g_i^0)^2} = |g_i^0|$$

$$\theta_i^2 \leftarrow \theta_i^1 - \frac{\eta}{\sigma_i^1} g_i^1 \quad \sigma_i^1 = \sqrt{\frac{1}{2} [(g_i^0)^2 + (g_i^1)^2]}$$

$$\theta_i^3 \leftarrow \theta_i^2 - \frac{\eta}{\sigma_i^2} g_i^2 \quad \sigma_i^2 = \sqrt{\frac{1}{3} [(g_i^0)^2 + (g_i^1)^2 + (g_i^2)^2]}$$

图4.1 sigma 更新的具体方法

RMSProp

$$\theta_i^{t+1} \leftarrow \theta_i^t - \boxed{\frac{\eta}{\sigma_i^t}} g_i^t$$

$$\theta_i^1 \leftarrow \theta_i^0 - \frac{\eta}{\sigma_i^0} g_i^0 \quad \sigma_i^0 = \sqrt{(g_i^0)^2} \quad 0 < \alpha < 1$$

$$\theta_i^2 \leftarrow \theta_i^1 - \frac{\eta}{\sigma_i^1} g_i^1 \quad \sigma_i^1 = \sqrt{\alpha (\sigma_i^0)^2 + (1 - \alpha) (g_i^1)^2}$$

$$\theta_i^3 \leftarrow \theta_i^2 - \frac{\eta}{\sigma_i^2} g_i^2 \quad \sigma_i^2 = \sqrt{\alpha (\sigma_i^1)^2 + (1 - \alpha) (g_i^2)^2}$$

图4.2 更好的方法 (alpha 为超参数)

- Adam 现在比较好的方法。
- Learning Rate Decay 学习率衰减 和时间有关系。
- Warm up 预热 学习率先变大 后变小

CNN

基本概念

是一种专门为处理具有网格结构数据（如图像、音频）而设计的深度学习模型。

是一种前馈神经网络，在图像识别、目标检测、语义分割等领域表现卓越。它通过卷积层提取数据中的特征，减少了对人工特征工程的依赖。

- 学习使用 git 工具克隆项目。

代码块

```
1 git clone https://github.com/用户名/仓库名.git
```

- 学习 conda

代码块

```
1 conda create -n cnn python=3.10 # 创建名为 cnn 的环境
2 conda activate cnn # 激活 cnn
3 conda deactivate # 退出当前环境
4 conda remove -n cnn --all # 删除环境
```

卷积层

卷积层是 CNN 的核心，它通过一组可学习的卷积核在输入图像上滑动，提取空间特征（如边缘、纹理、形状）。

- 卷积核大小： 3×3 、 5×5 等。
- 输入通道数：输入数据的“厚度”或“层数”。需要和上一层的输出通道数相等。
- 输出通道数：这一层输出特征图的数量。
- 填充：在输入图片的四周额外补上数值（通常是0），然后再进行卷积操作，防止越卷越小。

池化层

是 CNN 中用来压缩图片尺寸、突出显著特征的一种简单操作。它没有可学习的参数，只做筛选。

用于降低特征图的维度，减少计算量。常见的池化操作有最大池化和平均池化。

本代码采用最大池化层，只保留窗口里数值最大的那个像素，其余全部丢掉。

全连接层

全连接层位于CNN的最后部分，它将前面卷积层和池化层提取的特征进行整合，输出最终的分类结果或回归值。

- 输入单元数：由上一层决定。
- 输出单元数：自己决定，例如手写数字识别，0~9共十个数字，输出单元数即为10。

损失函数

用来优化模型，本代码采用交叉熵损失函数。

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

优化器

根据梯度去调整参数，下次做得更好。

前向传播

在神经网络中，前向传播是指数据从输入层开始，依次经过每一层（隐藏层、输出层）的计算，最终得到模型预测结果的过程。这个过程是单向流动的（输入 → 隐藏层 → 输出），因此称为“前向”。

- 第一个卷积层 -> ReLU 激活函数 -> 最大池化层
- 第二个卷积层 -> ReLU 激活函数 -> 最大池化层
- 将张量展平成一维向量
- 第一个全连接层 -> ReLU 激活函数
- 第二个全连接层，输出最终的分类结果

激活函数

- ReLU激活函数：

$$ReLU(x) = \max(0, x)$$

激活函数的作用是引入非线性，同时影响梯度传播、输出范围、稀疏性和训练稳定性。

反向传播

- 从损失 L 开始，计算损失对输出层输入的梯度。
- 利用链式法则，将这个梯度向后传递到隐藏层、再到输入层。
- 每传递一层，就计算出该层参数的梯度。

随机梯度下降算法 (SGD)

利用计算好的梯度、学习率（提前设定好的）、上一步的模型参数进行计算。

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t; x_i, y_i)$$

Transformer

1.Transformer 简介

1.是什么

Transformer 是一种完全基于 自注意力机制 (Self-Attention) 的神经网络结构。

它抛弃了此前主流的循环网络 (RNN、LSTM) 和卷积网络 (CNN) 的串行或局部处理方式，改为让序列中所有位置两两直接交互，从而一次性地建模全局依赖关系。

核心组件包括：

- 自注意力：让每个词在编码时都能“看到”整个句子，并根据与所有词的相关度生成新的表示。
- 多头注意力：用多组不同的注意力，让模型从不同角度捕捉语义。
- 位置编码：因为没有了循环和卷积，需要显式注入位置信息，使模型知道词的先后顺序。
- 编码器-解码器结构 (原始设计)：编码器理解输入，解码器生成输出。

2.用来做什么

自然语言处理 (机器翻译)、计算机视觉、多模态等。

3.比原先技术的优势

并行计算能力，训练速度极快

- 旧技术：RNN/LSTM 必须按时间步串行处理，第 t 步依赖第 $t-1$ 步的隐藏状态，无法并行。长序列训练非常慢，且难以充分利用 GPU。
- Transformer：自注意力一次性计算整个序列中所有位置的关系，训练时完全并行，训练效率提高，从而支持海量数据和大规模模型。

完美解决距离依赖问题

- 旧技术：RNN 随着序列增长，早期信息容易“遗忘”。RNN 天然带有邻近优先的归纳偏差，对近处关系敏感，但可能忽略远处信号。CNN 受限于卷积核感受野，堆叠多层才能看到全局。
- Transformer：任意位置无论多远，模型都能直接捕捉依赖（例如第 1 个词和第 100 个词的关系），这对理解长文本、生成连贯段落至关重要。

2.Transformer 具体结构及训练步骤

本代码是将德语翻译为英语的实现。

具体结构

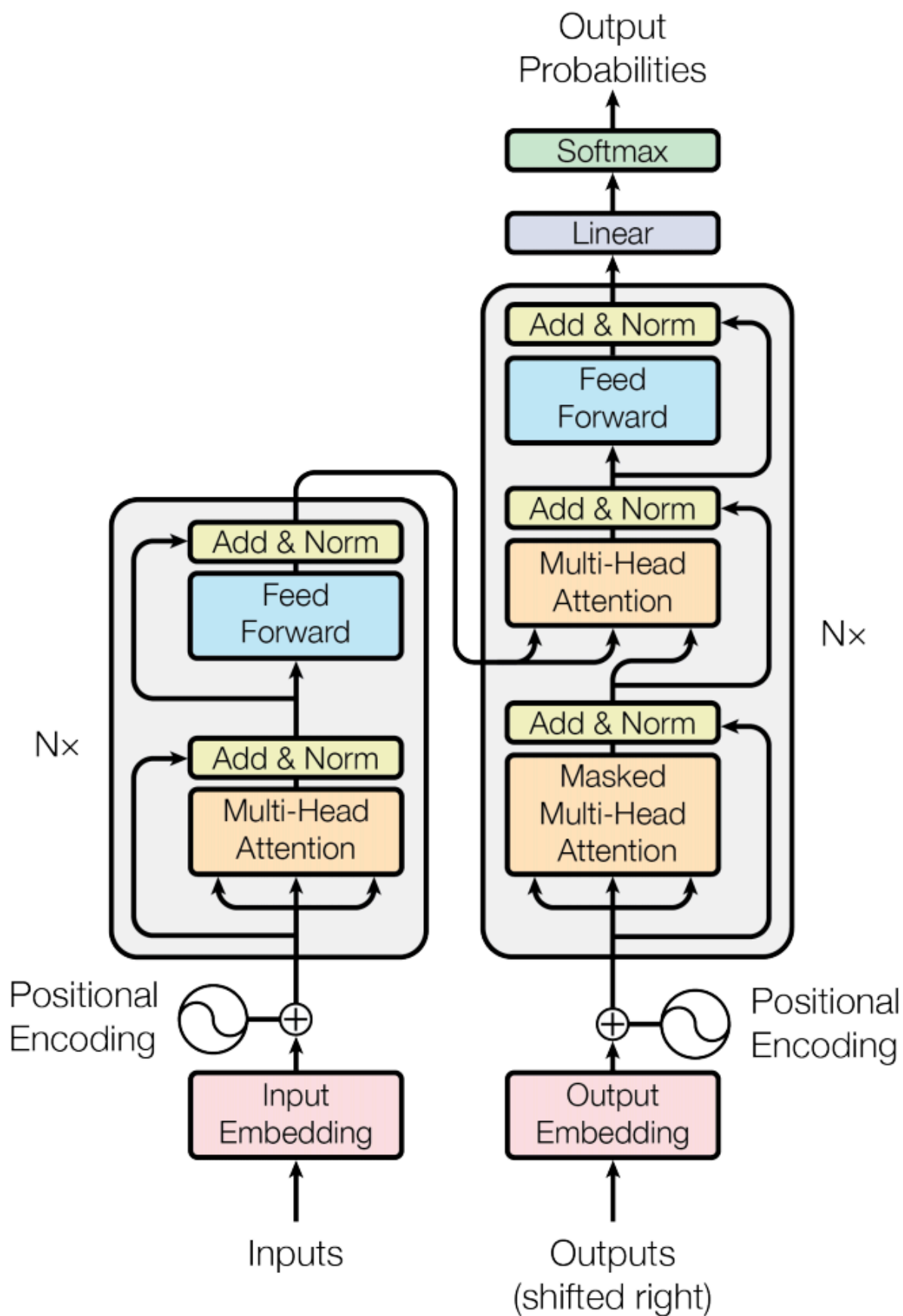


图2.1 Transformer 结构

训练过程，首先介绍一下 Encode 部分，也就是上图左边的一小块儿。

数据预处理

处理需要翻译的原句、预期结果，将一句话按照空格和标点分割，一个词就是一个 token，按照自己设定好的字典转化成对应的数字，变成一组向量。

例：'ich mochte ein bier P' (德语) 转变成 [1, 2, 3, 4, 0]。

例子中的 P 意思是 Padding，对于两个长短不一的句子，把短的句子填充上 padding，让两个张量形状一样，便于处理。

Embedding

原词嵌入、目标词嵌入。

把预处理之后的这组向量的每个元素经过线性变换再变成一组数据，原论文和代码实现一样都是512维。

现在上面举的例子变成了5x512的矩阵。

线性变换所用到的矩阵是模型的参数，首轮训练时候随机生成，之后反向传播每次更新。

位置编码

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

图2.2 位置编码所采用的函数

观察上面的函数，位置编码仅需要 pos (每个 token 的位置索引)、i (嵌入维度的索引)。

这两个变量和模型参数没有关系，所以这一步用不到模型参数，反向传播也不需要更新这个。

位置编码计算完之后，直接加到嵌入向量上。

多头注意力

(这一部分还使用了残差链接)。

上一步计算之后得到的嵌入向量成为这一步的 inputs。

将 inputs 通过线性变换，乘以模型参数 W_Q ，并且分成多个头，作为 Q。

将 inputs 通过线性变换，乘以模型参数 W_K ，并且分成多个头，作为 K。

将 inputs 通过线性变换，乘以模型参数 W_V ，并且分成多个头，作为 V。(这三个 inputs 是一样的都是最开始那个)

接着，生成了一些掩码，这一步具体用于让模型忽略掉自己在数据预处理时候填充的 padding。

接下来调用了缩放点注意力。

缩放点注意力

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

图2.3 缩放点注意力

除以一个 d_k 的平方根，防止内积过大，加速收敛。

做完之后，多个头都拼了回来。

前馈神经网络

再经过一个简单的前馈神经网络。

```
# ===== 前馈网络 =====
# 对应模型图中的 Feed Forward 和 Add & Norm
class PositionwiseFeedForward(nn.Module):
    def __init__(self):
        super(PositionwiseFeedForward, self).__init__()
        # 就是一个 MLP（两层全连接，中间 ReLU）
        self.fc = nn.Sequential(
            nn.Linear(d_model, d_ff),      # 第一层: d_model -> d_ff
            nn.ReLU(),                     # 激活函数
            nn.Linear(d_ff, d_model)       # 第二层: d_ff -> d_model
        )
```

图2.4 code

上述过程叠加好几层编码器之后就是 Encode 部分，接下来是 Decode 部分，也就是上面那个结构的右面一大块。

具体和上面有一些差别。

Tips：Decode 部分的输入是期望的翻译结果，Encode 部分的输入是原句子。

首先是交叉注意力，将 encode 部分的输出做为计算 K 和 V 的 inputs，计算 Q 时采用 decode 部分的 inputs。

因为 Q 和 K 会做向量乘积，这样就可以将原句子和目标句子联系起来。

又因为翻译时候逐词翻译，再结合上述的乘积过程，可以发现 Q 与 K 乘积的矩阵在 $j > i$ 时没有意义，所以通过掩码屏蔽掉。

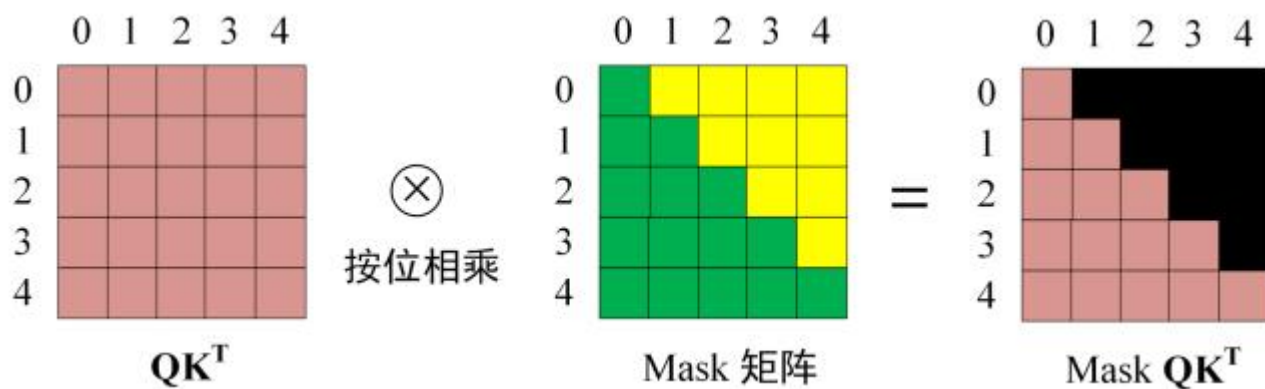


图2.5 掩码机制

其他区别不太大，最后得到的结果，经过一些投影变换，可以得到预测翻译单词的概率。

3.围绕 Transformer 的小巧思

样本的问题

```

sentence = [
    # 每一行是一个样本，包含三个字符串，空格分隔：
    # enc_input  原始输入的源语言句子（德语），末尾用'P'填充至统一长度
    # dec_input  解码器输入的目标语言句子，开头加上'S'起始符
    # dec_output 解码器的期望输出目标语言句子，末尾加上'E'结束符
    ['ich mochte ein bier P', 'S i want a beer .', 'i want a beer . E'],
    ['ich mochte ein cola P', 'S i want a coke .', 'i want a coke . E'],
]

```

图3.1 code

⚽ 这是测试时候的样例，观察 dec_input、dec_out，这两句话很像，那能不能一样呢？

答案是不能。

两个角度：

- 如果一样，模型在预测结果时候更倾向于复制输入的句子，因为训练时候输入输出一样，机智的 Transformer 会直接学到这个复制过程。
- 模型预测需要 S 标识符这个起始和 E 这个标识符作为结尾。

Q、K、V 实际意义的思考

从代码来看，我很难直接看出具体含义，论文给 Q、K、V 人为赋予了查询、键、值的含义。

根据缩放点注意力的函数，Q 和 K 做的是乘积，两者实际上可交换甚至是等价的，我猜只需要修改掩码部分的逻辑就可以实现 Q 和 K 的完全互换。

